CSBB 422 — BIG DATA ANALYTICS LAB PROJECT

COVID-19 Big Data Analysis System

Regional growth patterns, wave detection, and case-fatality correlation using Apache Hadoop MapReduce and Apache Spark on a distributed cluster.

COURSE

CSBB 422 — Big Data Analytics

STACK

Hadoop 3.2 + Spark 3.5 + Flask

DATASET

Our World in Data — COVID-19

CLUSTER

1 Master + 2 Worker Nodes

UI

Flask + Chart.js Dashboard

01 Project Overview

This project processes publicly available COVID-19 data on a distributed computing cluster. Raw CSV data is stored in **HDFS**, processed by **Hadoop MapReduce** (country-wise aggregation), and analysed by **Apache Spark** for trend detection, wave analysis, and correlation studies. Results are served through a **Flask web dashboard** with interactive Chart.js visualisations.

🗺️

Top 10 Countries by Cases

Cumulative totals ranked globally via MapReduce + Spark groupBy.

🌍

Continent Distribution

Share of global cases per continent — doughnut chart.

📈

Infection Wave Detection

7-day rolling average on global daily new cases.

💀

Case-Fatality Rate (CFR)

Deaths per 100 cases — highlights healthcare capacity differences.

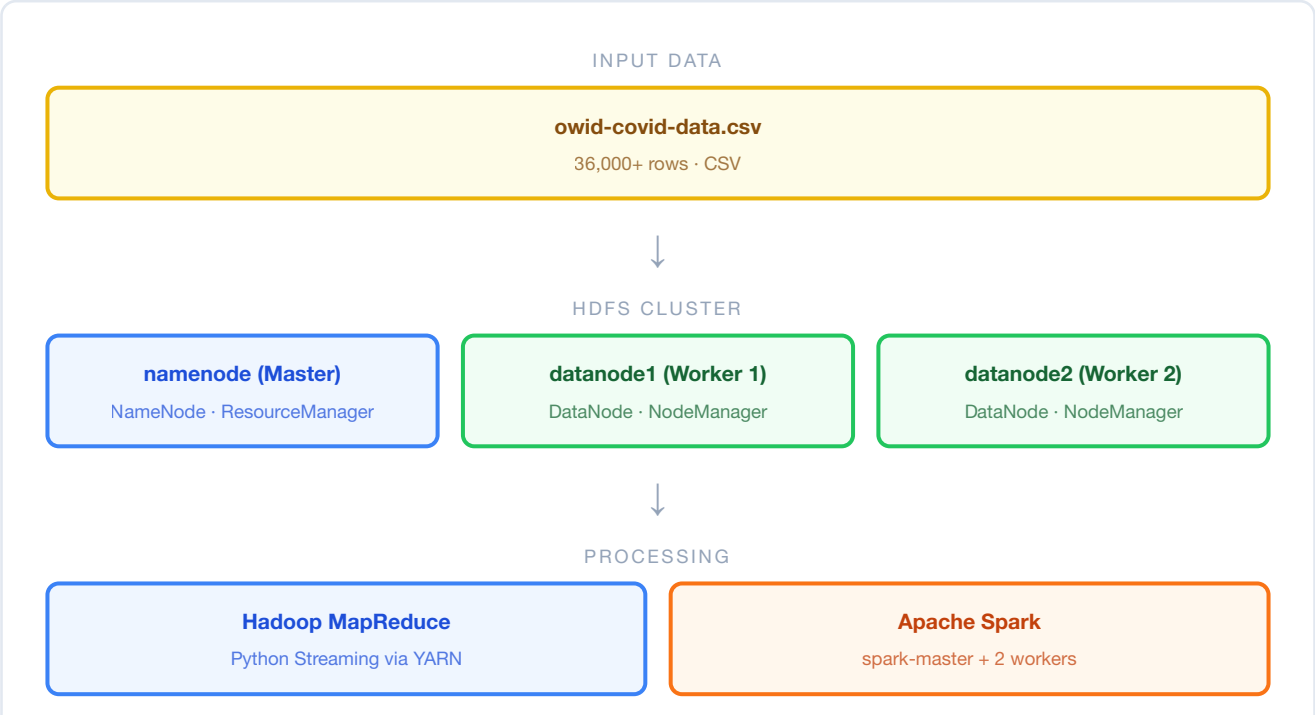
💉

Vaccination Progress

% population vaccinated (≥1 dose), ranked by country.

02 System Architecture

Three Docker containers simulate the physical 1-Master + 2-Worker cluster requirement. Each container communicates via a shared Docker bridge network using container hostnames.



↓

DASHBOARD

Flask Dashboard + Chart.js

http://localhost:5000

03 Project File Structure

```
covid19-analysis/
├── docker-compose.yml        # Full cluster: Hadoop + Spark + Dashboard
├── run_all.sh                # One-command Docker runner
│
├── config/                   # Hadoop XML config files
│   ├── core-site.xml           # HDFS default FS = hdfs://namenode:9000
│   ├── hdfs-site.xml           # Replication factor = 2
│   ├── mapred-site.xml         # MapReduce framework = yarn
│   ├── yarn-site.xml           # ResourceManager = namenode
│   └── workers                 # datanode1, datanode2
│
├── data/
│   ├── owid-covid-data.csv     # COVID dataset (36,000+ rows, 25 countries)
│   └── generate_sample.py      # Generates sample data if offline
│
├── mapreduce/
│   ├── mapper.py               # Emits: location\tcontinent\tnew_cases\tnew_deaths
│   └── reducer.py              # Sums per country → CSV output
│
├── spark/
│   └── covid_analysis.py       # 5 PySpark jobs → JSON results
│
├── dashboard/
│   ├── app.py                  # Flask API (6 routes)
│   ├── Dockerfile              # python:3.11-slim image
│   ├── requirements.txt        # flask==3.0.3
│   └── templates/index.html    # Chart.js dashboard (5 charts)
│
├── results/                  # JSON results written by Spark
│   ├── top10_countries.json
│   ├── continent_totals.json
│   ├── global_wave.json
│   ├── cfr_by_country.json
│   ├── vaccination_progress.json
│   └── mapreduce_output.csv
```

```
│
└── scripts/
    ├── 01_start_cluster.sh
    ├── 02_upload_data.sh
    ├── 03_run_mapreduce.sh
    ├── 04_run_spark.sh
    └── 05_run_local.sh          # No Docker — quick local demo
```

04 How to Run

Option A — Local Mode (No Docker, Quick Demo)

Requires only Python 3 and pip. Spark runs in local mode on one machine. Best for quick testing and demo.

```bash
# Step 1: Install Python dependencies
pip3 install pyspark flask pandas

# Step 2: Run everything (data → Spark → MapReduce → dashboard)
cd covid19-analysis
bash scripts/05_run_local.sh

# Step 3: Open dashboard in browser
#    http://localhost:5000
```

Option B — Docker Cluster (Full Multi-Node Demo)

Requires Docker Desktop installed. Launches 6 containers: NameNode, 2× DataNode, Spark Master, 2× Spark Worker, Dashboard.

```bash
cd covid19-analysis
bash run_all.sh
```

Step-by-Step (Docker Mode)

1 Start the Cluster

Launches all containers, waits for NameNode to become healthy.

```bash
bash scripts/01_start_cluster.sh
```

2 Upload Data to HDFS

Creates `/covid19/input/` in HDFS, uploads dataset.

```bash
bash scripts/02_upload_data.sh

# Verify the upload:
docker exec namenode hdfs dfs -ls /covid19/input/
```

3 Run Hadoop MapReduce Job

Python Streaming job via YARN. Aggregates total cases + deaths per country.

```
bash scripts/03_run_mapreduce.sh

# Test locally without cluster (pipe mode):
python3 mapreduce/mapper.py < data/owid-covid-data.csv \
  | sort | python3 mapreduce/reducer.py
```

4 Run PySpark Analysis (5 Jobs)

Submits to Spark cluster. Writes JSON result files to `/results/`.

```
bash scripts/04_run_spark.sh

# Run locally without Docker:
export COVID_DATA="$(pwd)/data/owid-covid-data.csv"
export COVID_RESULTS="$(pwd)/results"
python3 spark/covid_analysis.py
```

5 Open Dashboard

Flask server starts automatically on port 5000.

```
# URL:
http://localhost:5000

# Start manually if needed:
export COVID_RESULTS="$(pwd)/results"
python3 dashboard/app.py
```

05 Web UIs After Cluster Start

INTERFACE	URL	WHAT TO DEMONSTRATE
HDFS NameNode	`localhost:9870`	Live/Dead DataNodes, HDFS files under /covid19/
YARN ResourceManager	`localhost:8088`	MapReduce job history, YARN application list
Spark Master	`localhost:8080`	Connected workers, running Spark applications
Dashboard	`localhost:5000`	All 5 COVID-19 analysis charts

06 PySpark Jobs Explained

#	JOB NAME	SPARK OPERATION	OUTPUT FILE
1	Top 10 countries by total cases	`groupBy → max("total_cases") → orderBy desc → limit 10`	`top10_countries.json`
2	Continent-wise aggregation	`groupBy("continent") → sum("new_cases")`	`continent_totals.json`
3	Global infection wave (7-day avg)	`Window.rowsBetween(-3,3) → avg("daily_cases")`	`global_wave.json`
4	Case-fatality rate by country	`total_deaths / total_cases × 100`	`cfr_by_country.json`
5	Vaccination progress	`people_vaccinated / population × 100`	`vaccination_progress.json`

07 MapReduce Job Explained

Uses **Hadoop Streaming** — Python scripts read stdin, write to stdout. No Java required. YARN distributes the mapper across HDFS splits automatically.

```
DATA FLOW

HDFS: /covid19/input/owid-covid-data.csv
  |
  ▼  mapper.py  (runs on each HDFS split in parallel)
     Reads CSV rows, emits:
       "USA\tNorth America\t51523\t933"
       "IND\tAsia\t22001\t312"
  |
  ▼  Hadoop Sort + Shuffle  (sorts by country key)
  |
  ▼  reducer.py  (aggregates one country at a time)
     Sums new_cases, new_deaths per country:
       "USA,North America,89843211,1043221"
  |
  ▼  HDFS: /covid19/output/mapreduce/part-00000
```

Hadoop Streaming Command (on cluster)

```
docker exec namenode hadoop jar \
  /opt/hadoop-3.2.1/share/hadoop/tools/lib/hadoop-streaming-3.2.1.jar \
  -files /mapreduce/mapper.py,/mapreduce/reducer.py \
```

```
-mapper  "python3 mapper.py" \
-reducer "python3 reducer.py" \
-input   /covid19/input/owid-covid-data.csv \
-output  /covid19/output/mapreduce/
```

08 Dashboard API Routes

ROUTE	METHOD	RETURNS	POWERS CHART
`GET /`	HTML	Full dashboard page	—
`GET /api/top10`	JSON	Array of 10 countries	Horizontal bar
`GET /api/continents`	JSON	6 continents with totals	Doughnut
`GET /api/wave`	JSON	~200 daily data points	Line chart
`GET /api/cfr`	JSON	Top 15 CFR countries	Horizontal bar
`GET /api/vaccination`	JSON	Top 15 vaccinated	Horizontal bar

09 Dataset

Source: Our World in Data — `owid-covid-data.csv`

URL: `https://covid.ourworldindata.org/data/owid-covid-data.csv`

> **Offline fallback:** If internet is unavailable, run `python3 data/generate_sample.py` to generate 36,000 rows of realistic synthetic COVID-19 data for 25 countries, spanning Jan 2020 – Dec 2023.

COLUMN	TYPE	USED FOR
`iso_code`	string	Filter OWID_ aggregate rows (World, continents)
`continent`	string	Continent-level grouping
`location`	string	Country name (key for MapReduce)
`date`	date	Time-series axis (wave chart)
`new_cases`	integer	Daily new cases → wave, MapReduce sum
`new_deaths`	integer	Daily deaths → MapReduce sum, CFR
`total_cases`	integer	Top 10 ranking, CFR numerator

`total_deaths`	integer	CFR: total_deaths / total_cases × 100
`people_vaccinated`	integer	Vaccination %: vaccinated / population
`population`	integer	Vaccination % denominator

10 GitHub Repository Setup

The project must be submitted with a public GitHub repository link for evaluation.

BASH — One-time setup

```
# Initialize and push to GitHub
cd covid19-analysis
git init
git add .
git commit -m "feat: COVID-19 big data analysis — Hadoop + Spark + Flask"
git remote add origin https://github.com/YOUR_USERNAME/covid19-bigdata
git push -u origin main
```

Add .gitignore: Exclude `results/*.json`, `data/owid-covid-data.csv`, and `__pycache__/`. The evaluator runs the scripts fresh — don't commit generated output.

11 Project Requirements Checklist

REQUIREMENT	HOW MET
Install and configure Hadoop	Docker: bde2020/hadoop-namenode + datanode images, XML configs in config/
Install and configure Spark	Docker: bitnami/spark:3.5 — master + 2 workers
Basic usage commands	hdfs dfs -ls, dfsadmin -report, spark-submit, hadoop jar streaming
Working system using multiple technologies	HDFS + MapReduce + Spark + Flask + Chart.js all integrated
1 Master Node	namenode container (NameNode + ResourceManager + Spark Master)
2 Worker Nodes	datanode1 + datanode2 containers (DataNode + NodeManager + Spark Workers)
UI interface	Flask dashboard at localhost:5000 with 5 Chart.js visualisations
GitHub repository	git init + push (see Section 10)